arXiv:2508.19409v1 [math.NA] 26 Aug 2025

# Numerical Optimization for Tensor Disentanglement

Julia Wei[1], Alec Dektor[2], Chungen Shen[3], Zaiwen Wen[4], Chao Yang[2*]

[1]Department of Physics, Harvard University, 17 Oxford St., Cambridge, MA 02138, USA.
[2]Applied Mathematics and Computational Research Division, Lawrence Berkeley National Laboratory, 1 Cyclotron Rd, Berkeley, CA 94720, USA.
[3]College of Science, University of Shanghai for Science and Technology, 516 Jungong Rd, Shanghai, 200093, China.
[4]Beijing International Center for Mathematical Research, Peking University, 78 Jingchunyuan, Beijing, 100871, China.

*Corresponding author(s). E-mail(s): CYang@lbl.gov;
Contributing authors: juliawei@g.harvard.edu; adektor@lbl.gov; shenchungen@usst.edu.cn; wenzw@pku.edu.cn;

**Abstract**

Tensor networks provide compact and scalable representations of high-dimensional data, enabling efficient computation in fields such as quantum physics, numerical partial differential equations (PDEs), and machine learning. This paper focuses on tensor disentangling, the task of identifying transformations that reduce bond dimensions by exploiting gauge freedom in the network. We formulate this task as an optimization problem over orthogonal matrices acting on a single tensor's indices, aiming to minimize the rank of its matricized form. We present Riemannian optimization methods and a joint optimization framework that alternates between optimizing the orthogonal transformation for a fixed low-rank approximation and optimizing the low-rank approximation for a fixed orthogonal transformation, offering a competitive alternative when the target rank is known. To seek the often unknown optimal rank, we introduce a binary search strategy integrated with the disentangling procedure. Numerical experiments on random tensors and tensors in an approximate isometric tensor network

state are performed to compare different optimization methods and explore the possibility of combining different methods in a hybrid approach.

**Keywords:** Tensor networks, two-dimensional quantum states, tensor disentanglement, disentangler, Riemannian optimization, entanglement, gauge invariance

# 1 Introduction

Tensor networks are widely used in computational science to efficiently represent high-dimensional data and operators, enabling scalable algorithms for problems that would otherwise be intractable. A tensor network is a structured collection of multi-dimensional arrays (tensors) connected by products over shared indices. Prominent examples of tensor network structures include matrix product states (MPS) [1] or equivalently tensor trains (TT) [2], projected entangled pair states (PEPS) [3, 4], the multi-entanglement renormalization ansatz (MERA) [5, 6], as well as the canonical polyadic (CP), Tucker [7], and Hierarchical Tucker decompositions [8]. Tensor networks have been used as compact representations of large-scale data in several applications, e.g., quantum many-body systems [9], numerical methods for partial differential equations (PDEs) [10, 11], data analysis [12], and machine learning [13, 14].

A convenient diagrammatic notation for describing algorithms involving tensor networks is shown in Figure 1 [15, 16]. In these diagrams, each circle represents a tensor, and each connecting line denotes an index over which the two tensors are contracted. These connecting lines are called internal legs or bonds, and they encode the internal structure of the network. Lines that connect to only one tensor are called open legs, and they correspond to the indices of the overall tensor represented by the network. The number of lines emanating from a tensor indicates its dimension, and the number of open legs in the network indicates the dimension of the resulting global tensor. The size of each internal bond, i.e., the number of values it can take, is called the bond dimension, and it plays a key role in determining the representational power and computational cost of the network.

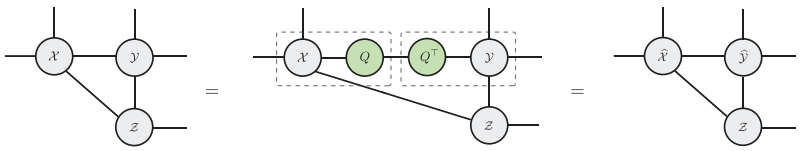



**Fig. 1** A tensor network consisting of three tensors $\mathcal{X}, \mathcal{Y}, \mathcal{Z}$. Applying a gauge transformation $Q$ to the tensor network results in a new, equivalent tensor network consisting of $\widehat{\mathcal{X}}, \widehat{\mathcal{Y}}, \mathcal{Z}$.

Many tensor networks have internal degrees of freedom that can be optimized to reduce the size of certain bonds, leading to a more efficient representation. When this is possible, we say that the tensor network can be *disentangled*. One example is inserting an invertible matrix and its inverse between two connected tensors. This

process, often referred to as a gauge transformation, leaves the tensor network invariant, though it can change the bond dimension when loops are present in the network. Figure 1 illustrates a gauge transformation, where an orthogonal matrix $Q$ and its inverse $Q^\top$ are inserted between tensors $\mathcal{X}$ and $\mathcal{Y}$. This transformation yields an equivalent tensor network in which $\widehat{\mathcal{X}}$ and $\widehat{\mathcal{Y}}$ are obtained by contracting $\mathcal{X}$ with $Q$ and $\mathcal{Y}$ with $Q^\top$, respectively. In this case, it may be possible to choose a $Q$ so that $\widehat{\mathcal{X}}$ can be decomposed into the contraction of two tensors $\widehat{\mathcal{U}}$ and $\widehat{\mathcal{W}}$ with lower bond dimension, e.g., using a truncated singular value decomposition, as shown in Figure 2, where the dimension across which the SVD is performed is indicated by a red dashed line. Contracting $\widehat{\mathcal{W}}$ with $\mathcal{Z}$ to form $\widehat{\mathcal{Z}}$ results in an equivalent tensor network with a reduced bond dimension between $\widehat{\mathcal{U}}$ and $\widehat{\mathcal{Z}}$.

Tensor disentanglers play a key role in a variety of tensor network algorithms. They are particularly important in classical simulations of quantum many-body systems, appearing in methods such as tensor network renormalization [6, 17], MERA [5, 18], purified MPS [19, 20], and isometric tensor network states (isoTNS) [21–23]. Disentanglers can also be used to decompose global unitary operators into sequences of unitary gates [24], enabling efficient implementation of matrix operations on quantum hardware, with applications in quantum machine learning [25]. Related optimization problems arise when determining basis transformations that reduce entanglement of a quantum state, e.g., Gaussian mode transformations in quantum chemistry [26], and determining coordinate transformations so that high-dimensional functions and operators admit low-rank tensor network representations [27, 28].

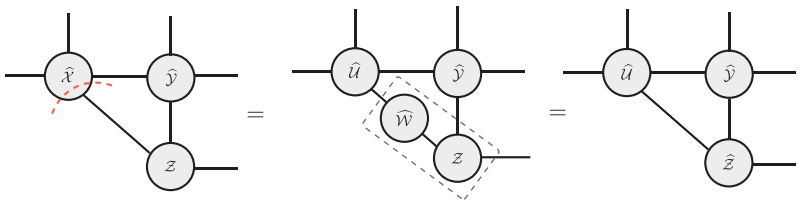



**Fig. 2** Performing a truncated SVD on $\hat{\mathcal{X}}$ across the dimension indicated by the red dashed line yields $\hat{\mathcal{U}}$ and $\hat{\mathcal{W}}$, which can have smaller dimensions than the bond connecting $\hat{\mathcal{X}}$ and $\mathcal{Z}$, provided the gauge transformation $Q$ shown in Figure 1 is chosen to disentangle $\hat{\mathcal{X}}$. Contracting $\hat{\mathcal{W}}$ and $\mathcal{Z}$ to obtain $\hat{\mathcal{Z}}$ yields a new tensor network with reduced bond dimension.

In this paper, we focus on a single tensor $\mathcal{X}$ within a tensor network, where bipartitioning the indices of $\mathcal{X}$ results in an unfolding matrix, $M$. We seek an orthogonal matrix $Q$ that, when contracted with $\mathcal{X}$ along a subset of indices, yields an $M(Q)$ with a low-rank approximation. Thus, $Q$ disentangles the system described by $\mathcal{X}$ along a particular bipartition.

We formulate the tensor disentangling task as a numerical optimization problem, highlighting the choice of different objective functions and incorporating the manifold constraint that restricts $Q$ to be an orthogonal matrix. The optimization

problem can be solved by first- and second-order Riemannian optimization methods, which require the computation of Riemannian gradients and Hessian-vector products. We detail efficient strategies for evaluating these quantities and analyze their computational complexity, as they represent the primary cost of Riemannian optimization.

In addition, we reformulate the disentangling problem as a joint optimization over $Q$ and a low-rank approximation of $M(Q)$, which can be solved via an alternating minimization algorithm. Through numerical experiments on isoTNS tensors, a subclass of PEPS representing two-dimensional quantum states, we demonstrate that this approach can outperform Riemannian methods in terms of efficiency. However, it assumes prior knowledge of the optimal rank of $M(Q)$, which is typically unknown in practice. To address this, we propose methods for finding an initial estimate of the rank and optimizing it iteratively using a binary search algorithm integrated with the disentangling procedure. Finally, through numerical experiments on random tensors, we show that combining Riemannian optimization with alternating minimization yields an efficient hybrid algorithm.

This paper is organized as follows. In the next section, we give a precise formulation of the tensor disentangling task as an optimization problem and introduce the notations used to define the objective function and constraints. Section 3 details a specific parameterization of orthogonal matrices that allows the task to be cast as an unconstrained optimization problem. We present Riemannian optimization methods [22, 29, 30] for solving the disentangling optimization problem in section 4. In section 5, we present the joint optimization framework and present the alternating minimization algorithm. We discuss some practical considerations in performing disentangling optimization in section 6. Numerical examples are given in section 7 to demonstrate the effectiveness of the methods proposed in this work and compare the efficiency of different methods for random and isoTNS tensors.

## 2 Problem Formulation and Notation

We consider a 4-dimensional tensor $\mathcal{X} \in \mathbb{R}^{l\times r\times b\times c}$. To describe the disentangling problem, we introduce a few operators. Let $\mathbf{M} : \mathbb{R}^{l\times r\times b\times c} \to \mathbb{R}^{lr\times bc}$ be the map that flattens a four-dimensional tensor into a matrix by combining the first two dimensions into rows and the last two dimensions into columns. Throughout, we let $X = \mathbf{M}(\mathcal{X}) \in \mathbb{R}^{lr\times bc}$. We denote the reverse mapping of a matrix of dimension $lr \times bc$ to a tensor of dimension $l \times r \times b \times c$ by $\mathbf{M}^{-1}$ so that $\mathcal{X} = \mathbf{M}^{-1}(X)$. Let $\mathbf{P}$ be a permutation that rearranges the dimensions of a tensor in the order $[1, 4, 2, 3]$ so that $\mathbf{P}(\mathcal{X}) \in \mathbb{R}^{l\times c\times r\times b}$. We use $\mathbf{P}^{-1}$ to denote the inverse of $\mathbf{P}$ so that $\mathbf{P}^{-1}(\mathbf{P}(\mathcal{X})) = \mathcal{X}$. Finally, we define composition map $\mathbf{A} = \mathbf{M}\circ\mathbf{P}\circ\mathbf{M}^{-1} : \mathbb{R}^{lr\times bc} \to \mathbb{R}^{lc\times rb}$, which is a unitary operator with inverse $\mathbf{A}^{-1} : \mathbb{R}^{lc\times rb} \to \mathbb{R}^{lr\times bc}$ defined as $\mathbf{A}^{-1} \equiv \mathbf{M}\circ\mathbf{P}^{-1}\circ\mathbf{M}^{-1}$. In Figure 3, we provide graphical depictions of these operators. The optimization algorithms described in the following sections can be trivially extended to higher-dimensional tensors by defining alternative reshaping and permutation operators.

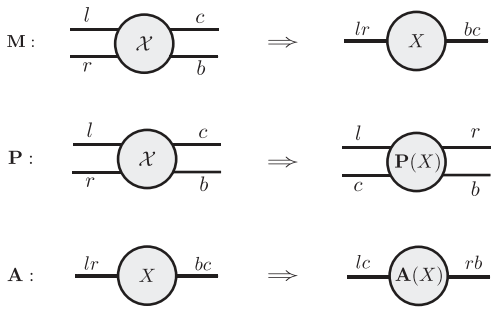



**Fig. 3** Tensor reshaping and permutation operations used to define the disentangler optimization problem.

We are interested in finding an orthogonal matrix $Q \in \mathbb{R}^{lr \times lr}$ that minimizes

$$c_k(Q) = \sum_{i=k+1}^{m} \sigma_i^2(\mathbf{A}(QX)) \tag{2.1}$$

for some $k > 1$ and $k \ll m$. Here $m = \min(lc, br)$ and the function $\sigma_i(\bullet)$ returns the $i$-th singular value of its input. The function in (2.1) measures the error of replacing $\mathbf{A}(QX)$ with its best rank-$k$ approximation. Equivalently, (2.1) measures the entanglement across a bond dimension of $\mathcal{X}$. When both (2.1) and $k$ are small, the matrix $\mathbf{A}(QX)$ can be replaced by a low-rank approximation, thereby reducing the total number of degrees of freedom in the tensor network representation of $\mathcal{X}$. In the physics literature, this procedure is often referred to as *disentanglement*, and $Q$ is referred to as a *disentangler*. A tensor network diagram of the disentangler problem is shown in Figure 4. Constraining $Q$ to be orthogonal ensures that $\mathbf{A}(QX)$ has the

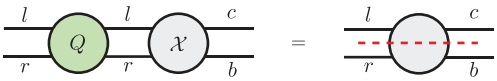



**Fig. 4** Tensor network diagram of the disentangler problem. The unitary operator $Q$ is to be optimized so that entanglement across the red dashed line is minimized.

same Frobenius norm as that of $\mathbf{A}(X)$, i.e.,

$$\sum_{i=1}^{m} \sigma_i^2(\mathbf{A}(QX)) = \sum_{i=1}^{m} \sigma_i^2(\mathbf{A}(X)). \tag{2.2}$$

Such a constraint is important so that (2.1) is minimized relative to the norm of the tensor $\mathcal{X}$ that is being disentangled. More broadly, we can consider objectives of the form

$$f(Q) = \sum_{i=1}^{m} \phi\left(\sigma_i(\mathbf{A}(QX))\right), \tag{2.3}$$

which depend on the choice of function $\phi : \mathbb{R} \to \mathbb{R}$. We can then formulate the disentangling of $\mathcal{X}$ as a constrained optimization problem

$$\min_{Q^\top Q = I} \sum_{i=1}^{m} \phi\left(\sigma_i(\mathbf{A}(QX))\right). \tag{2.4}$$

If $\phi(t) = \phi_\mu(t)$ is the piecewise function

$$\phi_\mu(t) = \begin{cases} 0 & \text{if } t \le \mu, \\ t^2 & \text{if } t > \mu, \end{cases} \tag{2.5}$$

for some thresholding parameter $\mu$ satisfying $\sigma_k < \mu < \sigma_{k+1}$, then the objective function in (2.4) is equivalent to (2.1). In this case, the disentangler $Q$ is chosen to minimize the error in the rank-$k$ approximation of $\mathbf{A}(QX)$.

When the parameter $\mu$ (or equivalently, the target disentanglement rank $k$) is not known a priori, other objective functions can be used to disentangle $\mathcal{X}$. One such function is the von Neumann entropy [31],

$$S_{\text{vN}} \equiv -\sum_{i=1}^{m} \sigma_i^2 \ln \sigma_i^2, \tag{2.6}$$

obtained by setting $\phi(t)$ to $\phi(t) = -t^2 \ln t^2$.

Another possibility is to choose $\phi(t) = t^{2\alpha}$ for some $\alpha \in (0, 1)$. Minimizing the objective function (2.3) obtained from such a $\phi(t)$ is equivalent to minimizing the Rényi-$\alpha$ entropy [32], defined by

$$S^\alpha \equiv \frac{1}{1-\alpha} \ln\Big(\sum_{i=1}^{m} \sigma_i^{2\alpha}\Big). \tag{2.7}$$

When $\alpha = 1/2$, minimizing $S^{1/2}$ is equivalent to minimizing $\sum_{i=1}^{m} \sigma_i$, which is the nuclear norm of $\mathbf{A}(QX)$, a well-known surrogate objective for the rank-minimization problem [33, 34].

Although we focus on real valued tensor in this work, the algorithms to be presented below can be easily extended to complex valued tensors, and the disentangler can be a complex and unitary matrix.

# 3 Unconstrained optimization

The constrained optimization problem (2.4) can be reformulated as an unconstrained optimization problem if we use a particular parameterization of the orthogonal

matrix $Q$. We discuss methods for solving such an unconstrained optimization problem in this section. In particular, we show how the gradient required in a first-order numerical optimization method can be computed.

Any orthogonal matrix can be parameterized as a matrix exponential, $Q = e^B$, where $B$ is a skew-symmetric matrix, i.e., $B = -B^\top$. If we collect all matrix elements in the strictly lower triangular part of $B$ into a vector $s$, and use $B(s)$ to represent the corresponding skew-symmetric matrix, then (2.4) can be formulated as an unconstrained optimization problem.

Substituting $Q = e^{B(s)}$ into (2.3) yields the objective function

$$g(s) \equiv f(e^{B(s)}). \tag{3.8}$$

It follows from the chain rule that the gradient of $g(s)$ can be evaluated as

$$\nabla g(s) = \frac{\partial f(Q)}{\partial Q} \cdot \frac{\partial Q}{\partial s}. \tag{3.9}$$

The derivative of $Q$ with respect to $s$ can be derived from a perturbative expansion of $e^{B+\Delta}$ for some $\Delta$. Because the directional derivative of $e^B$ along $\Delta$ contains nested commutators of $B$ and $\Delta$, the derivative does not have a simple and closed form, except in the case $s = 0$ in which it is simply the identity matrix.

The derivative of $f(Q)$ with respect to $Q$ can be evaluated by a closed-form expression given in the following theorem, which can also be found in [35]. This expression makes use of the singular value decomposition of $\mathbf{A}(QX)$.

**Theorem 3.1.** *If $\boldsymbol{A}(QX) = U(Q)\Sigma(Q)V(Q)^\top$ is the singular value decomposition, then the gradient of $f$ in (2.3) with respect to $Q$ is*

$$\nabla_Q f(Q) = \mathbf{A}^{-1}\left(U\phi'(\Sigma)V^\top\right)X^\top, \tag{3.10}$$

*where the derivative $\phi'$ of $\phi$ is applied entry-wise to $\Sigma$.*

*Proof* We begin by taking the directional derivative of (2.3) in the direction $E \in \mathbb{R}^{lr\times lr}$ and applying the chain rule to obtain

$$\begin{aligned} Df(Q)[E] &= \sum_{i=1}^{m} \phi'\left(\Sigma_{ii}\right) D\Sigma_{ii}(Q)[E] \\ &= \text{trace}\left(\phi'(\Sigma)D\Sigma(Q)[E]\right), \end{aligned} \tag{3.11}$$

where the second equality holds since $\phi'(\Sigma)$ is diagonal. Next, we obtain an expression for the directional derivative $D\Sigma(Q)[E]$. To do so, we differentiate $\mathbf{A}(QX)$

$$\begin{aligned} D\mathbf{A}(QX)[E] &= \mathbf{A}(EX) \\ &= \left(DU(Q)[E]\right)\Sigma V^\top + U\left(D\Sigma(Q)[E]\right)V^\top + U\Sigma\left(DV(Q)^\top[E]\right), \end{aligned} \tag{3.12}$$

where we used the fact that $\mathbf{A}$ is linear to obtain the first equality and the SVD representation of $\mathbf{A}(QX)$ with the product rule to obtain the second equality. Multiplying the preceding equation on the left by $U^\top$ and on the right by $V$ we obtain

$$D\Sigma(Q)[E] = U^\top \mathbf{A}(EX)V, \tag{3.13}$$

where we used the fact that $U$ and $V$ have orthonormal columns and that $U^\top\,(DU(Q)[E]) = 0$ and $\left(DV(Q)^\top[E]\right)V = 0$. Substituting (3.13) into (3.11) we obtain

$$\begin{aligned} Df(Q)[E] &= \text{trace}\left(\phi'(\Sigma)U^\top \mathbf{A}(EX)V\right) \\ &= \text{trace}\left(V\phi'(\Sigma)U^\top \mathbf{A}(EX)\right) \\ &= \left\langle U\phi'(\Sigma)V^\top, \mathbf{A}(EX)\right\rangle \\ &= \left\langle \mathbf{A}^{-1}\left(U\phi'(\Sigma)V^\top\right), EX\right\rangle \\ &= \left\langle \mathbf{A}^{-1}\left(U\phi'(\Sigma)V^\top\right)X^\top, E\right\rangle \end{aligned} \tag{3.14}$$

where we used the matrix inner product defined as $\langle X, Y\rangle = \text{trace}(X^\top Y)$ for any matrices $X, Y$ of the same dimension. The gradient $\nabla_Q f(Q)$ is the unique matrix satisfying $Df(Q)[E] = \langle \nabla_Q f(Q), E\rangle$ for all matrices $E$ of appropriate dimension. Comparing this with the last line in (3.14) completes the proof. □

The following algorithm summarizes the main steps for computing such a gradient, which will be utilized in numerical optimization algorithms for tensor disentanglement described in the following sections.

**Algorithm 1** Compute Euclidean gradient of disentangle cost function

**Require:**
$X \in \mathbb{R}^{lr\times bc} \to$ flattening of tensor $\mathcal{X}$ to be disentangled
$Q \in O(lr) \to$ disentangler at current iteration
**Ensure:** $\nabla_Q f(Q) \to$ Euclidean gradient of $f$ at $Q$
1: $Y = \mathbf{A}(QX)$
2: $[U, \Sigma, V] = \texttt{svd}(Y)$
3: $\nabla_Q f(Q) = \mathbf{A}^{-1}(U\phi'(\Sigma)V^\top)X^\top$

We estimate the computational complexity of Algorithm 3 assuming $lc \le rb$ for simplicity.

Step 1: Compute the matrix product $QX$, where $Q \in \mathbb{R}^{lr\times lr}$ and $X \in \mathbb{R}^{lr\times bc}$. This costs $\mathcal{O}((lr)^2 bc)$ FLOPs. The result is reshaped by the operator $\mathbf{A}$ into a matrix $Y \in \mathbb{R}^{lc\times rb}$.
Step 2: Perform an SVD of $Y \in \mathbb{R}^{lc\times rb}$. Since $lc \le rb$, the cost is $\mathcal{O}((lc)^2 rb)$ FLOPs.
Step 3: Apply the function $\phi'$ to the diagonal entries of $\Sigma$, and compute $U\phi'(\Sigma)V^\top$, which costs $\mathcal{O}((lc)^2 rb)$ FLOPs. Then, apply $\mathbf{A}^{-1}$ to reshape the result back to $\mathbb{R}^{lr\times bc}$ and multiply with $X^\top \in \mathbb{R}^{bc\times lr}$, costing $\mathcal{O}((lr)^2 bc)$ FLOPs.

Adding the costs from each step, the total computational complexity is $\mathcal{O}((lr)^2bc + (lc)^2rb)$.

We should note that the objective function defined in (2.4) is a function of a Löwner operator [36], which generates a matrix-valued function via applying a single-variable function to each of the singular values of a matrix. Properties of Löwner operators, including the differentiation of such operators, have been well studied in [35, 37, 38]. The gradient formula given in (3.10) can also be derived directly from the results shown in these works. In particular, the case in which the singular value $\sigma_k$ has a multiplicity greater than 1 is considered in [35]. In that case, $f(Q)$ is not differentiable and a Clark subdifferential can be used in a non-smooth optimization algorithm.

The Euclidean gradient expression given in (3.10) can be used in a first-order unconstrained optimization method. Starting from $s = 0$, we can use the gradient to update $s$. Using the updated $s$, we can update $X$ as $X \leftarrow QX$, where $Q = e^{B(s)}$. Applying $Q$ to $X$ is equivalent to resetting $s$ to 0. Hence, in the next unconstrained optimization step, we can use (3.10) again to compute the gradient and follow the steepest descent direction. This type of optimization is more conveniently formulated as a Riemannian optimization problem as we describe in the next section.

# 4 Riemannian optimization

Let $lr = n$. If we focus on the objective function $f(Q)$ referred to in Theorem 3.1 and restrict $Q$ to the embedded Riemannian submanifold $O(n)$ of $\mathbb{R}^{n\times n}$ consisting of all orthogonal matrices, we can formulate the disentangling problem (2.4) as a Riemannian optimization and solve it using first- and second-order Riemannian optimization methods [39, 40]. The idea of such methods is to traverse the manifold until we find a (local) solution to the optimization problem (2.4). To traverse the manifold, we need a notion of direction which is provided by the tangent spaces. Recall that at each point $Q \in O(n)$ the tangent space of the manifold $O(n)$ is

$$T_QO(n) = \{Q\Omega \in \mathbb{R}^{n\times n} : \Omega \in \text{Skew}(n)\}, \tag{4.15}$$

where $\text{Skew}(n)$ represents the set of all $n \times n$ skew symmetric matrices [40]. The tangent space (4.15) is a vector space that linearizes the manifold $O(n)$ around $Q$. We consider $O(n)$ as an embedded Riemannian submanifold of $\mathbb{R}^{n\times n}$, which means that every tangent space inherits an inner product $(\cdot,\cdot)$ from $\mathbb{R}^{n\times n}$. The collection of all such inner products defines a Riemannian metric, which allows us to adopt the notions of gradients and Hessians to the manifold $O(n)$. Effective numerical optimization on Riemannian manifolds relies on the efficient computation of Riemannian gradients and Hessians. A straightforward method for obtaining Riemannian gradients and Hessians on $O(n)$ utilizes the orthogonal projector $\text{Proj}_Q : \mathbb{R}^{n\times n} \to T_QO(n)$ onto the tangent space (4.15), defined by

$$\text{Proj}_Q(Z) = \frac{Z - QZ^\top Q}{2}. \tag{4.16}$$

Hereafter we derive expressions for the Riemannian gradient and Hessian of the disentangler cost function (2.3) and discuss their computational costs. Tensor diagrams of these expressions, as well as a derivation for complex manifolds, can be found in [41].

## 4.1 Riemannian gradient

The Riemannian gradient of $f$ at $Q$, denoted by $\text{grad} f(Q)$, belongs to the tangent space $T_Q O(n)$ and points in the direction of steepest ascent of the cost function $f$ on the manifold $O(n)$. To obtain $\text{grad} f(Q)$ we project the Euclidean gradient $\nabla_Q f(Q)$, defined by (3.10) in Theorem 3.1, orthogonally onto the tangent space (4.15). Applying such projection to $\nabla_Q f(Q)$ yields the Riemannian gradient

$$\text{grad} f(Q) = \text{Proj}_Q(\nabla_Q f(Q)) \tag{4.17}$$

To compute the Riemannian gradient we first compute the Euclidean gradient using Algorithm 3 and then apply the orthogonal projector (4.16) onto the tangent space. With the Riemannian gradient available, we can solve the disentangler problem (2.4) using first-order optimization algorithms on the manifold $O(n)$, such as gradient descent.

The computational cost of the Riemannian gradient can be broken into two parts. First, there is the cost of the Euclidean gradient, which scales as $\mathcal{O}((lr)^2 bc + (lc)^2 rb)$ as discussed in Section 3. Second, there is the cost of the orthogonal projection onto the tangent space defined in (4.17). The dominant cost in the projection is matrix multiplication between matrices of size $lr \times lr$, which scales as $\mathcal{O}((lr)^3)$. Thus the total cost of computing the Riemannian gradient scales as $\mathcal{O}((lr)^3 + (lr)^2 bc + (lc)^2 rb)$. To perform one iteration of gradient descent, one computes the Riemannian gradient, chooses a step-size, and then applies a retraction mapping that moves the current iterate on the manifold $O(n)$ in the descent direction.

## 4.2 Riemannian Hessian

The Riemannian Hessian of $f$ at $Q$, denoted by $\text{Hess} f(Q)$, is a symmetric linear map acting on the tangent space $T_Q O(n)$

$$\text{Hess} f(Q) : T_Q O(n) \to T_Q O(n). \tag{4.18}$$

Analogous to the Euclidean Hessian, the Riemannian Hessian characterizes how the Riemannian gradient varies as $Q$ varies along a tangent direction $E \in T_Q O(n)$. For the Riemannian embedded submanifold $O(n)$, the Riemannian Hessian can be computed by first computing the directional derivative of $\text{grad} f$ in the direction $E$ and then projecting the result onto the tangent space $T_Q O(n)$ using (4.16) [40]. Here, $\text{grad} f$ is a map which takes $Q \in O(n)$ to the Riemannian gradient (4.17).

Taking the differential of grad$f$ at $Q$ defined by (4.17) in the direction $E \in T_Q O(n)$ and using linearity of the differential we obtain

$$D\text{grad}f(Q)[E] = D\text{Proj}_Q(\nabla_Q f(Q))[E] = \frac{D\nabla_Q f(Q)[E] - D\left(Q\nabla_Q f(Q)^\top Q\right)[E]}{2}. \tag{4.19}$$

This first term in the numerator is the differential of the Euclidean gradient $\nabla_Q f(Q)$. Applying the product rule to the second term in the numerator of (4.19) we have

$$D\left(Q\nabla_Q f(Q)^\top Q\right)[E] = E\nabla_Q f(Q)^\top Q + Q\left(D\nabla_Q f(Q)[E]^\top\right)Q + Q\nabla_Q f(Q)^\top E, \tag{4.20}$$

which also requires the differential of the Euclidean gradient $\nabla_Q f(Q)$. The differential of the Euclidean gradient $\nabla_Q f(Q)$ (given in (3.10)) can be expressed as

$$\begin{aligned} D\nabla_Q f(Q)[E] &= D\left(\mathbf{A}^{-1}\left(U\phi'(\Sigma)V^\top\right)X^\top\right)[E] \\ &= \mathbf{A}^{-1}\left(DU[E]\phi'(\Sigma)V^\top + UD\phi'(\Sigma)[E]V^\top + U\phi'(\Sigma)DV^\top[E]\right)X^\top, \end{aligned} \tag{4.21}$$

where we used the fact that $\mathbf{A}^{-1}$ is linear and the product rule. The differentials of $U$, $\phi'(\Sigma)$ and $V$ in (4.21) we obtain using known results for differentiating the singular value decomposition [42]. Using the fact that $D\mathbf{A}(QX)[E] = \mathbf{A}(EX)$ we have

$$\begin{aligned} DU[E] &= U\left[F \odot \left(U^\top\mathbf{A}(EX)V\Sigma + \Sigma V^\top\mathbf{A}(EX)^\top U\right)\right] + \left(I_{lc} - UU^\top\right)\mathbf{A}(EX)V\Sigma^{-1} \\ D\phi'(\Sigma)[E] &= \phi''(\Sigma)D\Sigma[E] = \phi''(\Sigma)U^\top\mathbf{A}(EX)V \\ DV[E] &= V\left[F \odot \left(\Sigma U^\top\mathbf{A}(EX)V + V^\top\mathbf{A}(EX)^\top U\Sigma\right)\right] + \left(I_{rb} - VV^\top\right)\mathbf{A}(EX)^\top U\Sigma^{-1}, \end{aligned} \tag{4.22}$$

where

$$F_{ij} = \begin{cases} \dfrac{1}{\sigma_j^2 - \sigma_i^2}, & i \neq j \\ 0, & i = j, \end{cases} \tag{4.23}$$

$\odot$ denotes the Hadamard product and $I_m$ denotes the $m \times m$ identity matrix. Equations (4.19)-(4.23) provide a closed form expression for the differential $D\nabla_Q f(Q)[E]$ of the Euclidean gradient $\nabla_Q f(Q)$. The Riemannian Hessian of $f$ at $Q$ in the direction $E$ is now easily obtained by projecting the differential $D\text{grad}f(Q)[E]$ orthogonally onto the tangent space

$$\text{Hess}f(Q)[E] = \text{Proj}_Q\left(D\text{grad}f(Q)[E]\right). \tag{4.24}$$

We summarize the main steps for computing the Riemannian Hessian in the following algorithm.
We estimate the computational complexity of Algorithm 4.2 assuming $lc \leq rb$ for simplicity.

**Algorithm 2** Compute Riemannian Hessian of disentangle cost function

**Require:**
$X \in \mathbb{R}^{lr \times bc} \to$ flattening of tensor $\mathcal{X}$ to be disentangled
$Q \in O(lr) \to$ disentangler at current iteration
$E \in T_Q O(lr) \to$ tangent direction along which the Hessian is evaluated

**Ensure:** $\mathrm{Hess} f(Q)[E] \to$ Riemannian Hessian at $Q$ in the direction $E$

1: $Y = \mathbf{A}(QX)$
2: $[U, \Sigma, V] = \mathtt{svd}(Y)$
3: $\nabla_Q f(Q) = \mathbf{A}^{-1}(U\phi'(\Sigma)V^\top)X^\top$ ▷ Euclidean gradient
4: $\mathrm{grad} f(Q) = \mathrm{Proj}_Q\left(\nabla_Q f(Q)\right)$ ▷ Riemannian gradient
5: Evaluate $F$ in (4.23)
6: Evaluate $DU[E], D\phi'(\Sigma)[E], DV[E]$ in (4.22)
7: Evaluate $D\nabla_Q f(Q)[E]$ using (4.21)
8: Evaluate $D\mathrm{grad} f(Q)[E]$ using (4.19)
9: $\mathrm{Hess} f(Q)[E] = \mathrm{Proj}_Q\left(D\mathrm{grad} f(Q)[E]\right)$

Steps 1–4: Compute the Riemannian gradient costing $\mathcal{O}((lr)^3 + (lr)^2bc + (lc)^2rb)$, as discussed in Section 4.1.

Step 5: The cost of the $lc(lc-1)$ non-zero elements of $F \in \mathbb{R}^{lc \times lc}$ is $\mathcal{O}((lc)^2)$ which is negligible.

Step 6: First compute $\mathbf{A}(EX)$, where $E \in \mathbb{R}^{lr \times lr}$ and $X \in \mathbb{R}^{lr \times bc}$, which costs $\mathcal{O}((lr)^2bc)$. Then compute $U^\top \mathbf{A}(EX)V$, where $U \in \mathbb{R}^{lc \times lc}$, $\mathbf{A}(EX) \in \mathbb{R}^{lc \times rb}$, and $V \in \mathbb{R}^{rb \times lc}$, which costs $\mathcal{O}((lc)^2rb)$, and $V^\top \mathbf{A}(EX)^\top U$, which costs $\mathcal{O}((lc)^3)$. Diagonal scaling by $\Sigma, \phi''(\Sigma) \in \mathbb{R}^{lc \times lc}$ and Hadamard product with $F$ have a negligible cost of $\mathcal{O}((lc)^2)$. The total cost for $D\phi'(\Sigma)[E]$ and the first term in $DU[E], DV[E]$ is $\mathcal{O}((lc)^3 + (lr)^2bc + (lc)^2rb)$. Finally $\left(I_{lc} - UU^\top\right)\mathbf{A}(EX)V\Sigma^{-1}$ costs $\mathcal{O}((lc)^3 + (lc)^2rb)$ and $\left(I_{rb} - VV^\top\right)\mathbf{A}(EX)^\top U\Sigma^{-1}$ costs $\mathcal{O}((lr)^2bc + (lc)^2rb)$. The total cost in Step 6 is $\mathcal{O}((lc)^3 + (lr)^2bc + (lc)^2rb)$.

Step 7: The dominant costs are from multiplication involving matrices $U, DU[E], D\phi'(\Sigma)[E] \in \mathbb{R}^{lc \times lc}$ and $V, DV[E] \in \mathbb{R}^{lc \times rb}$, costing $\mathcal{O}((lc)^2rb)$, and multiplication of $\mathbf{A}^{-1}(\bullet) \in \mathbb{R}^{lr \times bc}$ with $X^\top \in \mathbb{R}^{bc \times lr}$, costing $\mathcal{O}((lr)^2bc)$. Diagonal scaling with $\phi'(\Sigma)$ has negligible cost. The total cost for Step 7 is thus $\mathcal{O}((lc)^2rb + (lr)^2bc)$.

Step 8: The difference of two matrices with size $lr \times lr$ and rescaling by $1/2$ has cost $\mathcal{O}((lr)^2)$, which is negligible.

Step 9: The dominant cost in the orthogonal tangent space projection is matrix multiplication between matrices of size $lr \times lr$, which scales as $\mathcal{O}((lr)^3)$.

Adding the costs from each step, the total computational complexity for the Riemannian Hessian is $\mathcal{O}((lr)^3 + (lc)^3 + (lr)^2bc + (lc)^2rb)$.

### 4.3 Riemannian nonlinear conjugate gradient and trust-region Newton methods

The Riemannian gradient defined in (4.17) can be used in a Riemannian nonlinear conjugate gradient (RCG) algorithm to obtain an optimal disentangler $Q^*$. In an RCG algorithm, the approximate $Q$ matrix is updated as

$$Q^{(j+1)} = \mathcal{R}(Q^{(j)}, t^{(j)}\Gamma^{(j)}), \tag{4.25}$$

where $\Gamma^{(j)}$ is a Riemannian search direction defined at the $j$-th approximation $Q^{(j)}$, $t^{(j)}$ is an appropriate step length, and $\mathcal{R}$ denotes a retraction operation that brings $Q^{(j)} + t^{(j)}\Gamma^{(j)}$, which is the update of $Q^{(j)}$ on the tangent space, back to the Riemannian manifold $O(n)$.

In RCG, a conjugate search direction $\Gamma$ is updated as

$$\Gamma^{(j)} = -\mathrm{grad} f(Q^{(j)}) + \beta^{(j)}\mathcal{T}(\Gamma^{(j-1)}), \tag{4.26}$$

where $\mathcal{T}$ denotes a parallel transport operator that maps a search direction in $T_{Q^{(j-1)}}O(n)$ to a search direction in $T_{Q^{(j)}}O(n)$, and the parameter $\beta^{(j)}$ is chosen to ensure that $\Gamma^{(j)}$ is approximately "conjugate" to the previous search direction with respect to the Hessian. In RCG, $t^{(j)}$ can be updated via an adaptive line search along the manifold [43], while $\beta^{(j)}$ can be determined by the Riemannian generalizations of the Fletcher-Reeves [44], Polak-Ribière-Polyak [45, 46], Hestenes-Stiefel [47], and other methods [48, 49]. The basic steps of an RCG algorithm is given in Algorithm 4.3. We refer readers to [43] for details on how $t^{(j)}$ and $\beta^{(j)}$ are computed, and how the parallel transport $\mathcal{T}$ and the retraction $\mathcal{R}$ are performed.

With the Riemannian Hessian available, we can use second-order Riemannian optimization methods. In step $j$ of these methods, one can move along a Newton search direction $\Gamma^{(j)}$ that satisfies the Newton correction equation

$$\mathrm{Hess} f(Q^{(j)})[\Gamma^{(j)}] = -\mathrm{grad} f(Q^{(j)}). \tag{4.27}$$

This is followed by a retraction step that places the new approximation back onto the manifold $O(n)$ [40].

Note that the search direction $\Gamma^{(j)}$ is within the tangent space associated with $O(n)$ at the $j$-th approximation $Q^{(j)}$. If $\mathrm{Hess} f(Q^{(j)})$ is positive definite, $\Gamma^{(j)}$ is guaranteed to be a descent direction. Taking the full Newton step yields the minimizer of the local quadratic approximation of $f(Q)$ over all $E \in T_{Q^{(j)}}O(n)$, which is defined as

$$m_{Q^{(j)}}(\mathcal{R}(E)) = f(Q^{(j)}) + \langle \mathrm{grad} f(Q^{(j)}), E\rangle + \frac{1}{2}\langle \mathrm{Hess} f(Q^{(j)})[E], E\rangle. \tag{4.28}$$

However, if the Riemannian Hessian is not positive definite but non-singular, the convergence of a naive implementation of the Newton's method can be problematic unless the initial guess of the minimizer is sufficiently close to the solution.

**Algorithm 3** A Riemannian conjugate gradient (RCG) algorithm.

**Require:**
$Q^{(0)} \in O(n) \rightarrow$ initial disentangler
`maxiter` $\rightarrow$ maximum number of iterations allowed
$\epsilon \rightarrow$ convergence tolerance

**Ensure:** $Q^*$ is a local minimizer of $f(Q)$ in $O(n)$.

$j = 0$
$\Gamma^{(0)} = \text{grad} f(Q^{(0)})$
converged = false
**while** (Not converged and $j <$ `maxiter`) **do**
  Compute a step length $t^{(j)} > 0$
  $Q^{(j+1)} \leftarrow \mathcal{R}(Q^{(j)}, t^{(j)}\Gamma^{(j)})$
  Compute $G^{(j+1)} = \text{grad} f(Q^{(j+1)})$
  **if** ($\|G^{(j+1)}\| \leq \epsilon$) **then**
    converged = true
    $Q^* = Q^{(j+1)}$
  **else**
    Compute $\beta^{(j+1)}$
    $\Gamma^{(j+1)} \leftarrow -G^{(j+1)} + \beta^{(j+1)}\mathcal{T}^{(j)}(\Gamma^{(j)})$ where $\mathcal{T}^{(j)} : T_{Q^{(j)}} \rightarrow T_{Q^{(j+1)}}$ is a parallel transport operator
  **end if**
  $j \leftarrow j + 1$
**end while**

One way to address convergence failure is to modify the Hessian by replacing it with a positive-definite operator. One widely used technique is to simply replace $\text{Hess} f(Q^{(j)})[E]$ on the left-hand side of (4.27) with

$$\text{Hess} f(Q^{(j)})[E] + \eta E, \tag{4.29}$$

where $\eta$ is an appropriately chosen *regularization* parameter to make the operator on the left-hand side of (4.27) positive definite [50].

Because $m_{Q^{(j)}}(\mathcal{R}(E))$ is a local approximation of $f(Q)$ on the tangent space defined at $Q^{(j)}$, taking a full Newton step may not be appropriate when the minimizer of $m$ is far from $Q^{(j)}$. One way to address this issue is to impose an additional constraint that defines a small region around $Q^{(j)}$ in which $m_{Q^{(j)}}(\mathcal{R}(E))$ can be trusted to be a good approximation of $f(Q)$. The search direction is obtained as the solution of the *trust-region* subproblem

$$\min_{\|E\| \leq \Delta} m_{Q^{(j)}}(\mathcal{R}(E)), \tag{4.30}$$

where $\Delta$ is a trust-region radius that is adjusted dynamically based on the ratio of the actual reduction in $f(Q)$ over the reduction in $m_{Q^{(j)}}$ when a optimal solution of (4.30) is used to update $Q^{(j)}$.

Because $\mathrm{Hess} f(Q^{(j)})$ is only available through its multiplication with a search direction $E$ as defined in (4.19)–(4.24), the Newton correction equation or the trust-region subproblem is often solved iteratively using a modified conjugate gradient method [51]. We refer readers to [40] on the algorithmic details of how these problems are solved in a trust-region Newton's method. The major steps of a Riemannian trust-region Newton's (RTRN) method are given in Algorithm 4.3.

**Algorithm 4** A Riemannian trust-region Newton (RTRN) algorithm.

**Require:**
$Q^{(0)} \in O(n) \to$ initial disentangler
$\Delta_{\max} \to$ maximum trust-region radius
$\Delta_0 \in (0, \Delta_{\max}) \to$ initial trust-region radius
`maxiter` $\to$ maximum number of iterations allowed
$r_{\min} \to$ minimum reduction ratio
$\epsilon \to$ convergence tolerance

**Ensure:** $Q^*$ is a local minimizer of $f(Q)$ in $O(n)$.

1: $j = 0$
2: $\Delta = \Delta_0$
3: converged = false
4: **while** (Not converged and $j <$ `maxiter`) **do**
5: Solve (4.30) and set the tentative next approximation to $\tilde{Q}^{(j+1)} = \mathcal{R}(E_{\text{opt}})$
6: Compute ratio $r = \frac{f(\tilde{Q}^{(j+1)}) - f(Q^{(j)})}{m(\tilde{Q}^{(j+1)}) - m(0)}$
7: Accept or reject $\tilde{Q}^{(j+1)}$:

$$Q^{(j+1)} = \begin{cases} \tilde{Q}^{(j+1)} & \text{if } r > r_{\min} \text{ (accept)}, \\ Q^{(j)} & \text{otherwise (reject)} \end{cases}$$

8: Update the trust region radius:

$$\Delta \leftarrow \begin{cases} \Delta/4 & \text{if } r < \frac{1}{4} \\ \min(2\Delta, \Delta_{\max}) & \text{if } r > \frac{3}{4} \text{ and } \|E_{\text{opt}}\| = \Delta \end{cases}$$

9: **if** ($\|\mathrm{grad} f(Q^{(j+1)})\| \leq \epsilon$) **then**
10: converged = true
11: $Q^* = Q^{(j+1)}$
12: **end if**
13: $j \leftarrow j + 1$
14: **end while**

# 5 Alternating optimization

As an alternative to the gradient-based optimization algorithms for computing the disentangler $Q$ described in the preceding sections, in this section we introduce an alternating least-squares scheme. We consider $\phi$ given in (2.5) and reformulate the optimization problem (2.4) as

$$\min_{(Q,M_k)\in O(lr)\times\mathbb{R}_k^{lc\times rb}} \|\mathbf{A}(QX) - M_k\|_F, \tag{5.31}$$

where $\mathbb{R}_k^{lc\times rb}$ is the manifold of rank-$k$ real matrices with size $lc\times rb$ and $O(lr)\times\mathbb{R}_k^{lc\times rb}$ is a product manifold. To see that (5.31) is equivalent to (2.4), notice that for any $Q$ the optimal rank-$k$ matrix $M_k$ is given by the rank-$k$ truncated SVD of $\mathbf{A}(QX)$. Fixing $M_k$ in (5.31) as the optimal truncated SVD yields an optimization problem for the disentangler $Q$

$$\min_{Q\in O(lr)} \sqrt{\sum_{i=k+1}^{m} \sigma_i^2(\mathbf{A}(QX))}, \tag{5.32}$$

which is equivalent to (2.4) with $\phi(t) = \phi_\mu(t)$ defined in (2.5). The advantage of considering (5.31) is that for fixed $Q$ we have a least-squares problem for $M_k$, and for fixed $M_k$ we have a least-squares problem for $Q$. Both of these least-squares problems have a simple, closed-form solution that is relatively cheap to compute. Alternating between these least-squares problems yields a sequence of disentanglers $Q^{(j)}$ for $j = 1, 2, \ldots$ that converges to a (local) minimum of (5.32).

To describe the alternating algorithm, we begin with a initial disentangler $Q^{(0)}$, e.g., the identity matrix $Q^{(0)} = I_{lr\times lr}$, to use as our zeroth iteration. We obtain iteration $Q^{(j+1)}$ from $Q^{(j)}$ as follows. Fix $Q^{(j)}$ in (5.31) and obtain $M_k^{(j)}$ by solving

$$M_k^{(j)} = \operatorname*{arg\,min}_{M_k\in\mathbb{R}_k^{lc\times rb}} \left\|\mathbf{A}\left(Q^{(j)}X\right) - M_k\right\|_F. \tag{5.33}$$

The preceding optimization problem is solved directly by taking the rank-$k$ truncated SVD $M_k^{(j)} = \texttt{SVD}_k\left(\mathbf{A}\left(Q^{(j)}X\right)\right)$. Then with $M_k^{(j)}$ fixed in (5.31) we obtain the updated disentangler $Q^{(j+1)}$ by solving

$$Q^{(j+1)} = \operatorname*{arg\,min}_{Q\in O(lr)} \left\|\mathbf{A}\left(QX\right) - M_k^{(j)}\right\|_F. \tag{5.34}$$

To obtain a closed form solution to (5.34), it is convenient to consider the equivalent problem

$$Q^{(j+1)} = \operatorname*{arg\,min}_{Q\in O(lr)} \left\|QX - \mathbf{A}^{-1}\left(M_k^{(j)}\right)\right\|_F, \tag{5.35}$$

known as an orthogonal Procrustes problem [52], which is solved by the polar decomposition

$$Q^{(j+1)} = UV^\top. \tag{5.36}$$

Here $U, V$ are the left, right factors in the SVD of the matrix $\mathbf{A}^{-1}\left(M_k^{(j)}\right)X^\top$. Iterating this procedure generates a sequence of disentanglers $Q^{(j)}$ and rank-$k$ matrices $M_k^{(j)}$ that reduce the objective function (5.31) monotonically and hence converge to a (local) minimum of the optimization problem (5.31). We remark that the Procrustes problem also appears in a disentangling algorithm that minimizes the Rényi-2 entropy [20]. More broadly, the alternating disentangler is a particular instance of Evenbly-Vidal optimization [17].

The main steps of the alternating disentangler algorithm are summarized below. To determine when the alternating disentangler algorithm has converged we consider

$$\delta Q^{(j+1)} = \left\| Q^{(j+1)} - Q^{(j)} \right\|_F, \tag{5.37}$$

which measures how much the disentangler has changed from iteration $j$ to $j+1$. Once (5.37) falls below a user-determined threshold, we say that the alternating algorithm has converged.

**Algorithm 5** Alternating disentangler algorithm

**Require:**
$X \in \mathbb{R}^{lr\times bc} \to$ flattening of tensor $\mathcal{X}$ to be disentangled
$Q^{(0)} \in O(lr) \to$ initial disentangler
$\texttt{maxiter} \to$ maximum number of iterations allowed
$\epsilon \to$ convergence tolerance

**Ensure:** $Q^* \in O(lr) \to$ local solution to (5.32)

1: $j = 0$
2: converged = false
3: **while** (Not converged and $j < \texttt{maxiter}$) **do**
4: $\quad M_k^{(j)} = \texttt{SVD}_k\left(\mathbf{A}(Q^{(j)}X)\right)$
5: $\quad [U, \Sigma, V] = \texttt{SVD}\left(\mathbf{A}^{-1}\left(M_k^{(j)}\right)X^\top\right)$
6: $\quad Q^{(j+1)} = UV^\top$
7: $\quad$ **if** $\left(\left\|Q^{(j+1)} - Q^{(j)}\right\|_F \le \epsilon\right)$ **then**
8: $\quad\quad$ converged = true
9: $\quad\quad Q^* = Q^{(j+1)}$
10: $\quad$ **end if**
11: $\quad j \leftarrow j+1$
12: **end while**

We estimate the computational complexity of a single iteration (Steps 3–5) of the alternating disentangler in Algorithm 5.

Step 3: Compute the matrix product between $Q^{(j)} \in \mathbb{R}^{lr\times lr}$ and $X \in \mathbb{R}^{lr\times bc}$ costing $\mathcal{O}((lr)^2bc)$. Then compute the rank-$k$ truncated SVD on $\mathbf{A}(Q^{(j)}X)$ costing $\mathcal{O}(klcrb)$.

Step 4: Compute the product of $\mathbf{A}^{-1}(M_k^{(j)}) \in \mathbb{R}^{lr\times bc}$ and $X^\top \in \mathbb{R}^{bc\times lr}$ costing $\mathcal{O}((lr)^2bc)$. Then compute the SVD of $\mathbf{A}^{-1}(M_k^{(j)})X^\top \in \mathbb{R}^{lr\times lr}$ costing $\mathcal{O}((lr)^3)$.

Step 5: The disentangler is computed as the matrix product $UV^\top$ where $U, V \in \mathbb{R}^{lr\times lr}$ costing $\mathcal{O}((lr)^3)$.

Adding the costs from each step, the total computational complexity of a single iteration of the alternating disentangler is $\mathcal{O}((lr)^3 + (lr)^2bc)$.

# 6 Practical Issues

We discuss a few practical issues to be considered in the implementation of the disentangling optimization algorithms.

## 6.1 Rank initialization and determination

Minimizing the objective function (2.1) by Riemannian or alternating optimization algorithms requires us to provide a rank parameter $k$ in (5.31). Ideally, we would like $k$ to be as small as possible while keeping the optimized objective function in (5.31) sufficiently small. However, such a minimal rank is generally unknown a priori. We will show that it is possible to obtain an estimate of the minimal rank by either counting the number of degrees of freedom in the disentangled tensor representation of $X$.

Suppose the disentangler $Q$ allows us to achieve

$$\sum_{i=k+1}^{m} \sigma_i^2(\mathbf{A}(QX)) = 0, \tag{6.38}$$

for some $k$. This implies we can find a $Q$ parameterized by $e^B$, where $B$ is a skew-symmetric matrix such that $M_k = \mathbf{A}(QX)$ is an $lc \times rb$ rank-$k$ matrix. We can parameterize $M_k$ as the product of $Y$ and $W^\top$, where $Y \in \mathbb{R}^{lc\times k}$ and $W \in \mathbb{R}^{rb\times k}$. Clearly, $Y$, $W$, and $Q$ satisfy the equation

$$Q^\top \mathbf{A}^{-1}(YW^\top) = X. \tag{6.39}$$

If the matrix elements in the lower- (or upper-) triangular part of $B$ as well as those in $W$ and $Y$ are treated as unknowns, the solution of (6.39) may exist if the number of unknowns matches the number of (scalar) equations, which is $lrbc$.

A naive sum of the degrees of freedom on the left-hand side of (6.39) includes $klc$ and $krb$ degrees of freedom in $Y$ and $W$, respectively, and $lr(lr-1)/2$ degrees of freedom in $B$ (recall $Q = e^B$), yielding $k(lc + rb) + \frac{lr(lr-1)}{2}$.

However, we should note that orthogonal matrices of the form

$$Q = Q_l \otimes Q_r,$$

where $Q_l$ and $Q_r$ can be represented as $e^{B_l}$ and $e^{B_r}$ for some $l \times l$ and $r \times r$ skew-symmetric matrices $B_l$ and $B_r$ respectively, do not contribute to the disentangling of $\mathcal{X}$. Therefore, we should exclude the number of degrees of freedom in $Q_l$ and $Q_r$, which are $l(l-1)/2$ and $r(r-l)/2$ respectively. Furthermore, the choices of $W$ and $Y$ are not unique because $YW^\top = YSS^{-1}W^\top = \hat{Y}\hat{W}^\top$, where $\hat{Y} = YS$ and $\hat{W} = WS^{-\top}$, for any non-singular matrix $S$. Hence, we need to remove $k^2$ degrees of freedom from the gauge matrix $S$ also. (For completeness, if $lr > bc$, we have to subtract the additional gauge degrees of freedom corresponding to an orthogonal matrix of dimension $(lr - bc) \times (lr - bc)$; below, we consider $lr \leq bc$ only, which is the case for most applications.)

As a result, when $lr \leq bc$, the total number of degrees of freedom on the left-hand side of (6.39) is

$$k(lc + rb) - k^2 + \frac{lr(lr-1)}{2} - \frac{l(l-1)}{2} - \frac{r(r-1)}{2}. \tag{6.40}$$

This number must be at least $lrbc$, which is the number of degrees of freedom in $X$ (and the number of scalar equations in (6.39)), to ensure (6.39) can be solved in general.

As an example, let us first consider an equal-dimensional $l \times l \times l \times l$ tensor. To ensure (6.39) is solvable, we should choose a minimal $k$ to satisfy

$$2l^2k - k^2 + \frac{l^2(l^2-1)}{2} - l(l-1) \geq l^4. \tag{6.41}$$

For $l \geq 2$, it follows that the optimal rank $k_l^*$ can be evaluated as

$$k_l^* = \lceil l^2 - \frac{\sqrt{2}}{2}\sqrt{(l-1)^2 l(l+2)} \rceil \tag{6.42}$$

For large $l$, $k_l^* \approx \frac{2-\sqrt{2}}{2} l^2 + O(l) \approx 0.29l^2$.

Second, we consider the case in which $l = r$ and $b = c$. For $c \geq l \geq 2$, the optimal rank $k_{l,c}^*$ is

$$k_{l,c}^* = \lceil lc - \frac{\sqrt{2}}{2}\sqrt{(l-1)^2 l(l+2)} \rceil \tag{6.43}$$

In the limit where $l \ll c$, we observe that $k_{l,c}^*$ is nearly equal to $lc$, up to a rank reduction of $\sim \frac{\sqrt{2}}{2} l^2$. This matches our expectation that a small disentangler is ineffective if further assumptions are not made about $X$.

It should be noted that the rank estimate determined by (6.40) does not take into account the numerical value of each element in $X$ or the accuracy requirement of the low-rank approximation. Therefore, the estimated rank can be larger than the optimal one. Nonetheless, the estimate given by Equation 6.40 can serve as a starting point for choosing the rank when benchmarking different optimization methods. For example, by choosing $k \geq k^*$, we can test how well different optimizers find an approximate solution to Equation 6.38 by minimizing Equation 2.1.

### 6.2 Rank minimization

We say that $Q$ is a good rank-$k$ disentangler if

$$c_k(Q) \leq \epsilon, \tag{6.44}$$

for some user-provided truncation error tolerance $\epsilon$. However, as we mentioned earlier, we would also like $k$ to be as small as possible. Therefore, in addition to solving the optimization problem (2.4) or (5.31), we would like to seek an optimal $k$. We propose to identify such a minimal $k$ by using a binary search algorithm given in Figure 6.2. In this algorithm, we initialize left and right bounds for searching for the optimal $k$ at $k_l$ and $k_r$. They can be set to $0$ and $\min(lc, rb)$ if no better bounds are known; in practice, we optimize a surrogate function such as the Rényi-1/2 entropy (2.7) and evaluate $c_k$ at the resulting disentangler to initialize $k_r$.

The next estimate of the optimal rank is taken to be $k = \lfloor (k_l + k_r)/2 \rfloor$. For each $k$, we use either a Riemannian optimization or the alternating minimization algorithm to find an optimal disentangler $Q_k$. We then examine the value of $c_k(Q_k)$. If $c_k(Q_k) \leq \epsilon$, we set $k_r$ to $k-1$. Otherwise, we set $k_l$ to $k+1$. While this ensures that the algorithm will terminate when $k_l > k_r$, we note that $k_r$ is not a strict upper bound. As a result, we update the optimal upper bound variable $k_{\text{opt}} = k$ whenever (6.44) is satisfied, which is returned when the algorithm terminates.

### 6.3 Local minima

We also note that disentangler optimization is a highly non-convex problem, irrespective of the chosen objective function. To illustrate the topography of the objective landscape, we consider a tensor $\chi \in \mathbb{R}^{4\times4\times4\times4}$ with uniformly distributed random entries in $[0, 1]$. We normalize $X$ with respect to the Frobenius norm and consider the objective (2.3) with $\phi$ chosen as the Heaviside function (2.5) and $\mu$ corresponding to rank-10 truncation error, i.e., $\sigma_{10} < \mu < \sigma_{11}$. We set all but two entries of the vector $s$ equal to zero and in Figure 5 plot the objective as a function of two non-zero entries $s_1, s_2 \in [-2\pi, 2\pi]$. The plot shows multiple local minima within this two-dimensional parameter space. While the local minima differ slightly in value, they are all sufficiently close to one another that, for practical purposes, any of them would yield a suitable disentangler.

## 7 Numerical examples

We test the optimization of disentanglers on two types of tensors. The first is a random tensor with entries drawn from a standard Gaussian distribution. The second is taken from an approximate isometric tensor network state (isoTNS) representation [21] of the ground state of the two-dimensional transverse-field Ising (2D TFI) Hamiltonian, $H = -J \sum_{\langle i,j \rangle} Z_i Z_j - g \sum_i X_i$, on an $L \times L$ lattice with open boundary conditions. IsoTNS is a tensor network ansatz with isometric constraints that enable efficient contraction and manipulation of 2D quantum states. An important procedure for isometric tensor networks is moving the orthogonality center, which can be

**Algorithm 6** Binary search rank selection and disentangler optimization

**Require:**

$X \in \mathbb{R}^{lr \times bc} \to$ flattening of tensor $\mathcal{X}$ to be disentangled

$Q^{(0)} \in O(lr) \to$ initial disentangler

$\epsilon \to$ truncation error tolerance

$\texttt{maxiter} \to$ maximum number of disentangler optimization routines

**Ensure:**

$k_{\text{opt}} \to$ truncation rank

$Q \to$ disentangler such that truncation error is bounded by $\epsilon$, i.e., $c_k(Q) \leq \epsilon$

$k_l \leftarrow 0$

$Q \leftarrow$ RCG disentangler optimization with Rényi-1/2 objective (2.7)

Determine $k_r$ such that $c_{k_r}(Q) \leq \epsilon$

$k_{\text{opt}} \leftarrow k_r$

$\texttt{iter} \leftarrow 1$

**while** $k_l \leq k_r$ and $\texttt{iter} < \texttt{maxiter}$ **do**

  $k \leftarrow \lfloor (k_l + k_r)/2 \rfloor$

  $Q_k \leftarrow$ Alternating or RCG optimization of rank-$k$ truncation error objective $c_k(Q)$.

  **if** $c_k(Q_k) \leq \epsilon$ **then**

    $k_{\text{opt}} \leftarrow k$

    $Q \leftarrow Q_k$

    $k_r \leftarrow k - 1$

  **else**

    $k_l \leftarrow k + 1$

  **end if**

  $\texttt{iter} \leftarrow \texttt{iter} + 1$

**end while**

return $k_{\text{opt}}, Q$

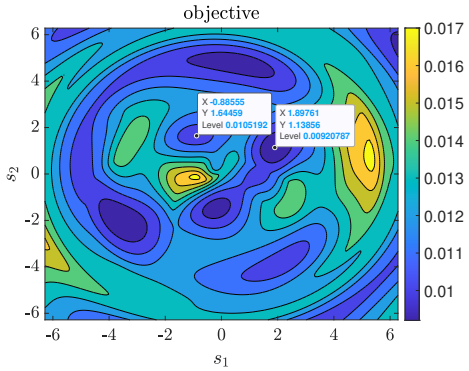



**Fig. 5** Objective function (2.3) measuring rank-10 truncation error for a $4 \times 4 \times 4 \times 4$ random tensor plotted over two skew-symmetric degrees of freedom $s_1, s_2 \in [-2\pi, 2\pi]$.

done using the Moses Move [21]. The Moses Move consists of a sequence of isometric tensor ring decompositions [53], which rewrites a region of the network in a form that shifts the orthogonality center. Disentangler optimization is a central step in the isometric tensor ring decomposition and directly impacts its accuracy and cost. Our second test tensor comes from an isoTNS approximation of the TFI ground state with $L = 8$, $g = 3.5$, $J = 1$, computed using a 2D generalization of time-evolving block decimation with imaginary-time step size $d\tau = 0.025$, bond dimension $\chi = 12$, and orthogonality hypersurface bond dimension $\chi' = 20$ [21, 22].

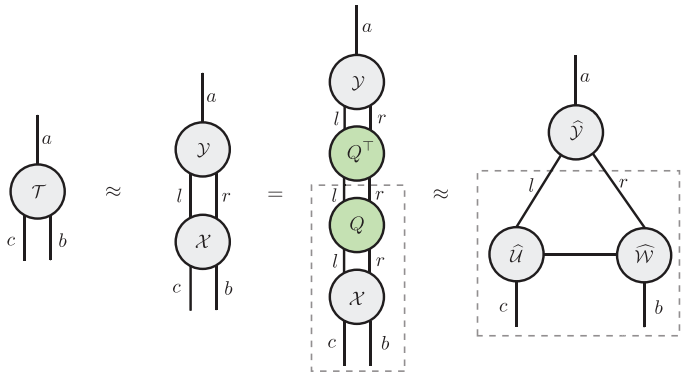



**Fig. 6** Isometric tensor ring decomposition. A three-dimensional tensor is decomposed using SVD with the resulting bond split into two dimensions indicated by $l$ and $r$. Then an optimized disentangler is inserted to minimize entanglement in $\mathcal{X}$. A second SVD completes the isometric tensor ring decomposition.

Hereafter we provide numerical comparisons of objective functions and optimization algorithms described in the preceding sections to our test tensors. All numerical experiments were performed using codes written in Python. We use the Pymanopt software to perform Riemannian optimization [54]. Timing measurements were collected using AMD "Milan" CPUs, with computing resources from the National Energy Research Scientific Computing Center.

## 7.1 Comparing different algorithms

We first compare different optimization methods for minimizing the objective function (2.1) with a fixed $k$. The methods we compare include the alternating disentangler (Alt), Riemannian conjugate gradient (RCG), and Riemannian trust-region Newton (RTRN). For the RTRN algorithm, we try both the unregularized Hessian, and a regularized Hessian with the regularization parameter $\eta = 10^{-12}$ in (4.29) (RTRN-reg). We use both the random and isoTNS tensors as test problems. For the equal-dimensional random tensor with size $n = 12$, we set $k$ to 44 (following (6.42)). For the isoTNS tensor, we set $k$ to 8. In all numerical experiments, the maximum number of iterations is set to 10,000 for Alt, 4,000 for RCG, and 1,000 for RTRN. The

initial guess for the disentangler is set to the identity matrix $I_{lr}$. To determine convergence for Alt, the minimum change in the disentangler norm, $\delta Q$, is set to $10^{-12}$; in addition, we set the minimum change in the objective function norm to $10^{-12}$. To determine convergence for RCG and RTRN, the minimum norm of the Riemannian gradient is set to $10^{-8}$; the RCG method for updating the parameter $\beta^{(i)}$ is Hestenes-Stiefel.

In Figure 7, we plot the change of the truncation error $c_k$ with respect to the iteration number for a few hundred iterations. We observe that, for the random tensor, RTRN has the fastest convergence rate, whereas the Alt exhibits the slowest convergence. The convergence rate of RCG is between the two, which is expected. The convergence of the regularized RTRN behaves similarly to that of RTRN, but truncation error is ultimately limited by the regularization parameter.

For the isoTNS tensor, the truncation error is quickly reduced from $9 \times 10^{-4}$ to $8 \times 10^{-5}$ within a few hundred iterations of RTRN and Alt. Both algorithms appear to stagnate at this truncation error level from that point on, although Alt yields a slightly smaller truncation error. The convergence of RCG is slower and it stagnates at a higher truncation error.

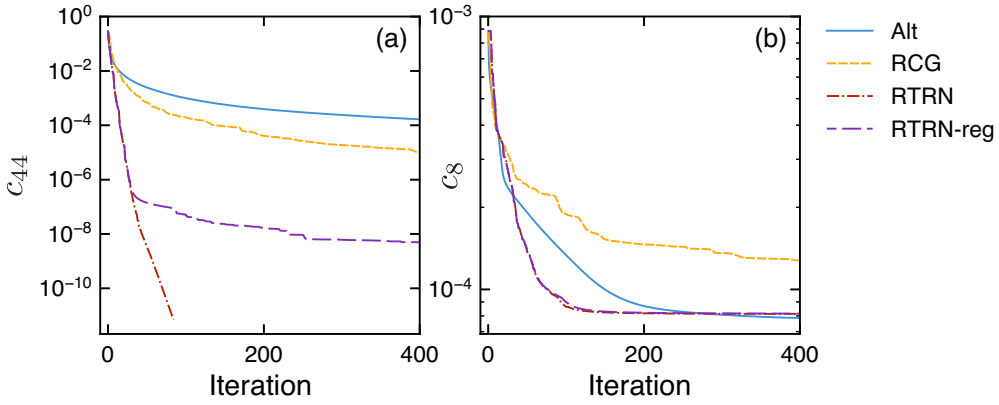



**Fig. 7** Iteration count of different methods for optimizing the truncation error at fixed rank $k$. (a) A random tensor of size (12, 12, 12, 12) with $k = 44$. (b) An isoTNS tensor of size (8, 8, 60, 12) with $k = 8$.

Although the iteration count of each algorithm shows their convergence rate, it does not fully reflect the computational cost of the algorithm, which also depends on the cost per iteration. A single (outer) iteration of the RTRN algorithm contains many inner conjugate gradient micro-iterations (set to a maximum of 100 inner iterations), and is thus more costly than a single iteration of RCG or Alt.

In Figure 8, we compare the efficiency of different algorithms by plotting the truncation error against the wall clock time, corresponding to roughly thousands of Alt and RCG iterations. In the random case, we find that for a given truncation error, RCG and RTRN converge comparably quickly; including Hessian information does not seem to improve the convergence efficiency. While the alternating disentangler

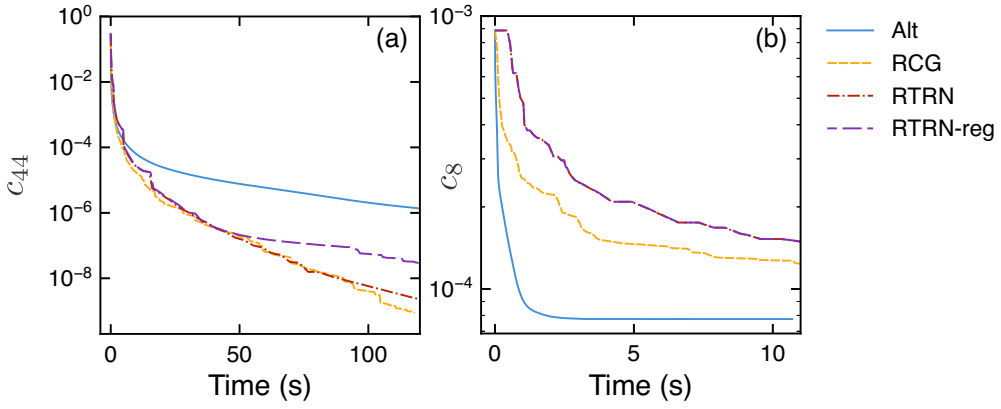



**Fig. 8** Wall time of different methods for optimizing the truncation error at fixed rank $k$. (a) A random tensor of size (12, 12, 12, 12) with $k = 44$. (b) An isoTNS tensor of size (8, 8, 60, 12) with $k = 8$.

performs better for the first few iterations, it seems to eventually stagnate. In contrast, in the isoTNS case, the alternating disentangler converges the fastest to the lowest truncation error. Figure 9(b) shows that Alt yields a visibly faster decay in singular values when disentangling this type of tensor. For either type of tensor, we do not find an improvement in convergence when the Hessian is regularized.

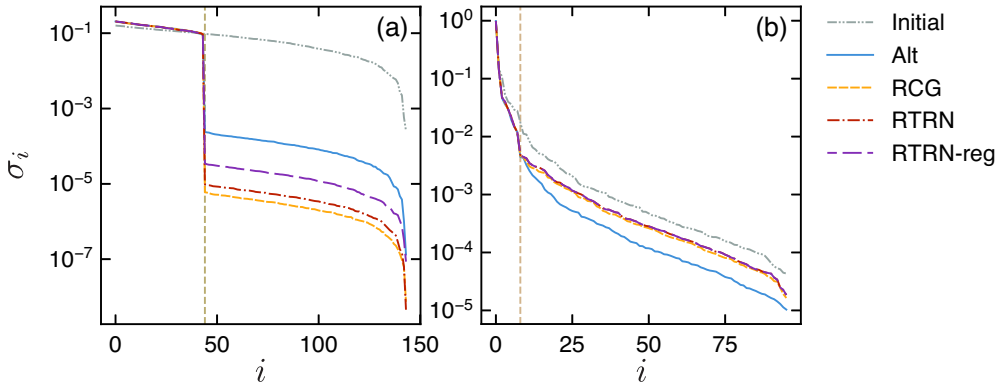



**Fig. 9** Singular value spectrum of different methods after optimizing the truncation error at fixed rank $k$, indicated by the vertical dotted lines. The spectra for different algorithms are taken at approximately the same wall time (the maximum timescales in Figure 8). (a) A random tensor of size (12, 12, 12, 12) with $k = 44$ at time $t = 118$ s. (b) An isoTNS tensor of size (8, 8, 60, 12) with $k = 8$ at $t = 11$ s.

## 7.2 Hybrid optimization method

We combine different optimization algorithms in order to test whether a hybrid scheme can lead to a more efficient method. For example, we can use the alternating

disentangler first, and then switch to either the RCG or RTRN method as shown in Figure 10. For random tensors, we observe that beginning with several iterations of the alternating disentangler before switching to RCG results in a similar convergence time as RCG, while switching to RTRN may lead to lower efficiency than RTRN. For the isoTNS tensor, where the alternating disentangler had the best performance, we observe that switching to either RCG or RTRN after starting with Alt does not improve convergence, and instead results in stagnation of the cost function. For this case, we observe that the norm of the gradient fluctuates around $10^{-4}$ or $10^{-5}$ in both RCG and RTRN (insets of Figure 10(b, d) depending on the number of Alt iterations taken prior to switching to RCG or RTRN, potentially due to the presence of many local minima. Since it does not rely on the gradient, the alternating disentangler seems to escape these local minima more efficiently. These results suggest that for optimizing an unknown tensor, starting with many iterations of the Alt disentangler, followed by RCG if Alt does not converge, may be an appropriate hybrid scheme.

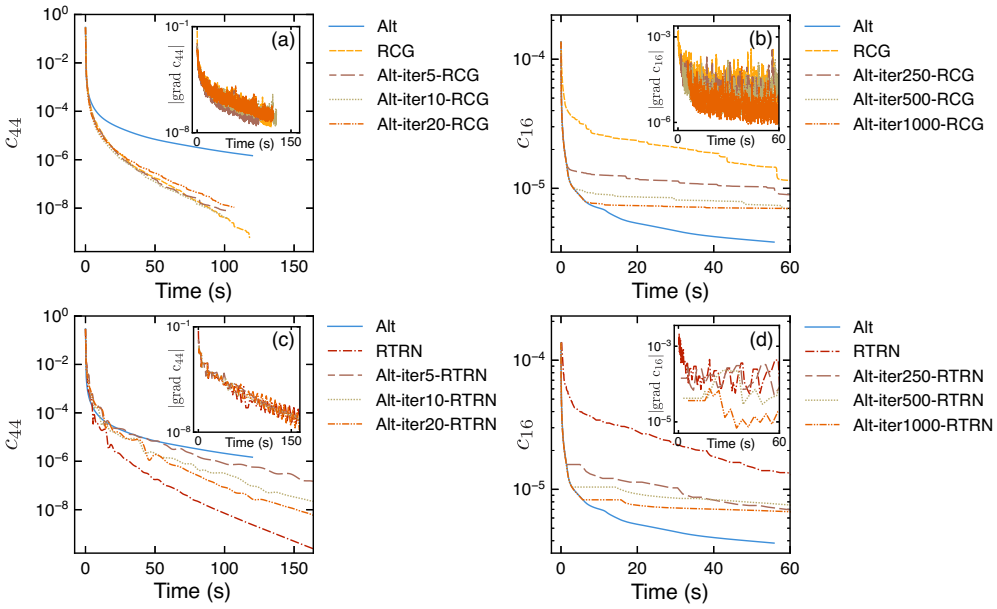



**Fig. 10** Wall time of different hybrid schemes for optimizing the truncation error at fixed rank $k$, where the inset shows the norm of the gradient during a representative run. (a) Timings for Alt with varied iterations (5, 10, 20), followed by RCG for a random tensor of size (12, 12, 12, 12) with $k = 44$. The primary figure shows averaged timing results over five random tensors. (b) Timings for Alt with varied iterations (250, 500, 1,000), followed by RCG for an isoTNS tensor of size (8, 8, 60, 12) with $k = 16$. (c) Timings for Alt with varied iterations (5, 10, 20), followed by RTRN for random tensors of the same size as (a). (d) Timings for Alt with varied iterations (250, 500, 1,000), followed by RTRN for an isoTNS tensor, as in (b).

| iter | $k$ | $k_l$ | $k_r$ | $k_{\text{opt}}$ | $c_k$ |
|---|---|---|---|---|---|
| 1 | 45 | 0 | 89 | 89 | $2.35 \times 10^{-11}$ |
| 2 | 22 | 0 | 44 | 45 | $2.39 \times 10^{-2}$ |
| 3 | 34 | 23 | 44 | 45 | $2.08 \times 10^{-3}$ |
| 4 | 40 | 35 | 44 | 45 | $1.24 \times 10^{-4}$ |
| 5 | 43 | 41 | 44 | 45 | $1.94 \times 10^{-6}$ |
| 6 | **44** | 44 | 44 | 45 | $\mathbf{8.80 \times 10^{-10}}$ |

**Table 1** Binary search results for minimizing the rank of a random tensor of size (12, 12, 12, 12) using CG optimization, given a truncation error tolerance $\epsilon = 10^{-6}$. The final value of $k_{\text{opt}}$ and the corresponding cost $c_{k_{\text{opt}}}$ are highlighted in bold.

### 7.3 Rank minimization

As we discussed earlier, the minimal rank $k_{\text{opt}}$ that can be achieved by a disentangler depends on the amount of truncation error $c_{k_{\text{opt}}}(Q)$ we can tolerate. The value of $k_{\text{opt}}$ is generally unknown in advance. In this section, we show how such a minimal rank can be determined for a given truncation error tolerance $\epsilon$ using the binary search method (Algorithm 6.2) presented in section 6.

Algorithm 6.2 requires an initial estimation of the minimal rank as an input. We can use either the estimate obtained from (6.40), which is derived from counting the number of degrees of freedom, or perform a disentanglement optimization using one of the surrogate objective functions such as the von Neumann ($S_{\text{vN}}$) and Rényi-$\alpha$ ($S^{\alpha}$) entropies, defined in (2.6) and (2.7), respectively. Once we obtain an optimal disentangler $Q$, we can then choose a $k$ that satisfies $c_k(Q) \leq \epsilon$.

In Figure 11, we plot the truncation error (2.1) versus truncation rank $k$ for disentanglers $Q = I$, $Q = Q_{\text{vN}}$, and $Q = Q_{\text{Rényi}}$, where $Q_{\text{vN}}$ and $Q_{\text{Rényi}}$ are obtained by minimizing the von Neumann and Rényi-1/2 entropy objective functions using the RCG algorithm. We can see from Figure 11(a) that, for the random tensor, the truncation error of $c_k(Q_{\text{vN}})$ decreases slowly with respect to $k$. The rate of change in truncation error $c_k(Q_{\text{vN}})$ is similar to that of $c_k(I)$, although $c_k(Q_{\text{vN}})$ is slightly below $c_k(I)$ for each $k$. This indicates that $\sigma_i(\mathbf{A}(QX))$ decreases slowly with respect to $i$ for this choice of $Q$, which is shown in Figure 12(a).

When the Réyi-1/2 entropy $S^{1/2}$ is chosen as the objective function, $c_k(Q_{\text{Rényi}})$ starts to decrease more rapidly when $k$ is close to 50, and becomes less than $10^{-20}$ when $k$ is slightly less than 100. Clearly, $Q_{\text{Rényi}}$ is a much better disentangler for the random tensor. Using the error tolerance $\epsilon = 10^{-6}$, we obtain the initial estimate $k = 89$. Using this initial rank estimate as the input to the binary search algorithm, we can obtain the minimal rank $k_{\text{opt}} = 44$ in six steps. Table 1 shows the change of rank and the corresponding $c_k$. We observe that this tolerance yields a rank estimate that matches Equation (6.42), and that the truncation error rapidly increases once $k < 44$. If we were to use the degrees of freedom count, which provides an initial rank estimate of $k^* = 44$, we note that one fewer binary search is needed to reach the optimal rank $k_{\text{opt}}$.

| iter | $k$ | $k_l$ | $k_r$ | $k_{\text{opt}}$ | $c_k$ |
|---|---|---|---|---|---|
| 1 | 19 | 0 | 38 | 38 | $1.99 \times 10^{-6}$ |
| 2 | 29 | 20 | 38 | 38 | $4.44 \times 10^{-7}$ |
| 3 | **24** | 20 | 28 | 29 | $\mathbf{8.25 \times 10^{-7}}$ |
| 4 | 22 | 20 | 23 | 24 | $1.23 \times 10^{-6}$ |
| 5 | 23 | 23 | 23 | 24 | $1.04 \times 10^{-6}$ |

**Table 2** Binary search results for minimizing the rank of an isoTNS tensor of size (8, 8, 60, 12) using alternating disentangler optimization, given truncation error tolerance $\epsilon = 10^{-6}$. The final value of $k_{\text{opt}}$ and the corresponding cost $c_{k_{\text{opt}}}$ are highlighted in bold.

In contrast, for the isoTNS example, the initial tensor already has a relatively fast decay in singular values. In Figure 11(b), we instead observe that the values of $c_k(Q)$ are comparable for $Q = Q_{\text{vN}}$ and $Q = Q_{\text{Rényi}}$. Since $b, c \gg l, r$ in the isoTNS case, the minimal $k$ that makes (6.40) no less than $lrbc$ for $l = r = 8$, $b = 60$ and $c = 12$ is $k^* = 91$. This is close to the full rank of $\mathbf{A}(\mathbf{X})$, and a poor initial guess of the minimal rank given the error tolerance $\epsilon = 10^{-6}$. Therefore, we use the initial rank estimate obtained from the optimal $Q_{\text{Rényi}}$ as the input to Algorithm 6.2 to seek the minimal rank. Starting with an initial rank estimate of $k = 38$, Table 2 shows that a few iterations are sufficient to find a minimal rank of $k_{\text{opt}} = 24$. We observe in Figure 11(b) that compared to optimizing the Rényi-1/2 objective, using Alt to optimize (5.32) with a target rank of $k_{\text{opt}} = 24$ further reduces the truncation error by a factor of 6.

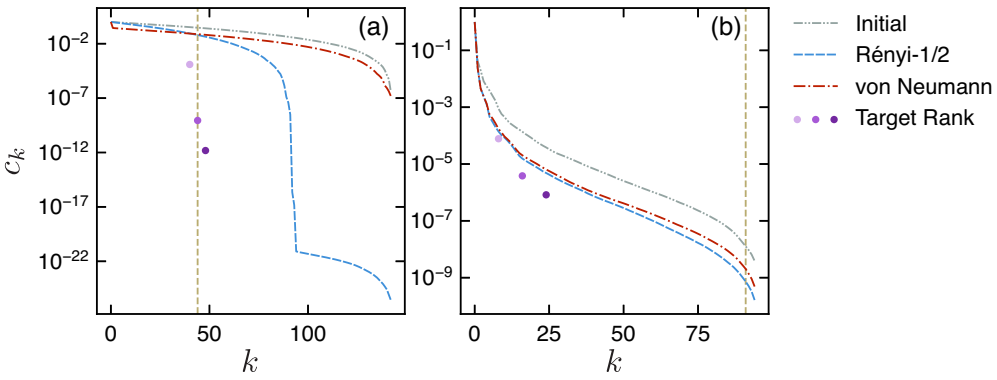



**Fig. 11** Comparison of different objective functions for rank estimation. The von Neumann and Rényi-1/2 objectives are optimized using RCG. The vertical dotted lines indicate $k^*$ (Equation 6.40). (a) A random tensor of size (12, 12, 12, 12), with target rank values $k \in \{40, 44, 48\}$. The truncation error objective is optimized using RCG. (b) An isoTNS tensor of size (8, 8, 60, 12), with target rank values $k \in \{8, 16, 24\}$. The truncation error objective is optimized using Alt.

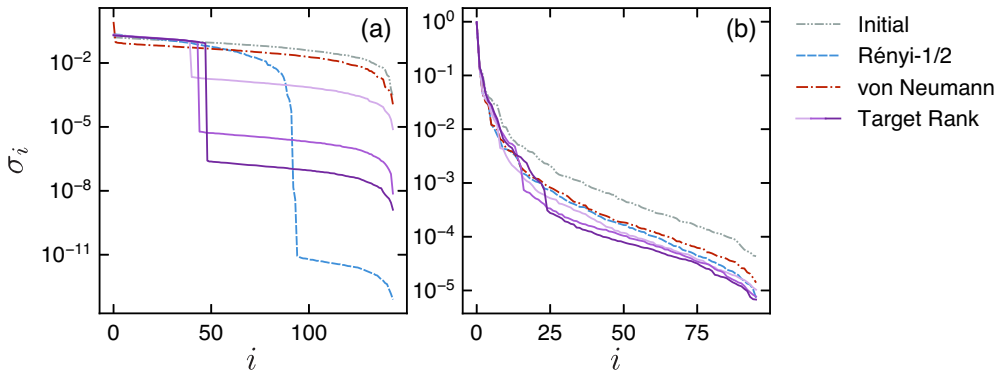



**Fig. 12** Comparison of the singular value distribution after optimization of different objective functions, using the same methods as Figure 11. (a) A random tensor of size (12, 12, 12, 12). (b) An isoTNS tensor of size (8, 8, 60, 12).

# 8 Conclusion

In this work, we have formulated the tensor disentanglement problem within the framework of Riemannian optimization. We discussed the choice of different objective functions and showed how both the Riemannian gradient and Hessian-vector products can be computed efficiently for first- and second-order Riemannian optimization algorithms. We have also shown that the disentangling task can be formulated as a joint optimization over orthogonal and low-rank matrices, and solved by an alternating minimization algorithm with lower per-iteration complexity compared to Riemannian optimization methods. Practical issues such as choosing an initial rank estimate and minimizing the rank via a bisection algorithm were also addressed.

Numerical experiments were conducted on both synthetic random and application-driven tensors. The results show that while the Riemannian trust-region Newton method achieves the fastest convergence in terms of the number of outer iterations, its high per-iteration cost limits its practical utility. In contrast, the Riemannian conjugate gradient algorithm is more efficient for random tensors, whereas alternating minimization is the fastest for structured tensors arising from physics applications. Furthermore, we demonstrated how the bisection strategy can effectively identify the minimal rank, with different rank initialization strategies.

Looking forward, we aim to improve the practical viability of second-order methods by developing effective preconditioners to accelerate the solution of the Newton correction equation. Additionally, we plan to investigate alternative constraints on the disentangler in order to enhance the quality of entanglement reduction across the tensor network and broaden the applicability of the proposed framework.

**Acknowledgements.** The authors wish to thank S. Anand, S.-H. Lin, Y. Wu, M. P. Zaletel, N. Y. Yao, C. Zhang, and A. Seigal for helpful discussions about running

isoTNS algorithms and the existence of exact disentanglers. J. W. acknowledges support by the U.S. Department of Energy Computational Science Graduate Fellowship under award number DE-SC0022158. This work was also supported in part by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Scientific Discovery through Advanced Computing (SciDAC) Program through the FASTMath Institute under U.S. Department of Energy Contract No. DE-AC02-05CH11231. (A. D. and C. Y.)

## References

[1] Perez-Garcia, D., Verstraete, F., Wolf, M.M., Cirac, J.I.: Matrix product state representations. Quantum Info. Comput. **7**(5), 401–430 (2007)

[2] Oseledets, I.V.: Tensor-train decomposition. SIAM Journal on Scientific Computing **33**(5), 2295–2317 (2011) https://doi.org/10.1137/090752286

[3] Orús, R.: A practical introduction to tensor networks: Matrix product states and projected entangled pair states. Annals of Physics **349**, 117–158 (2014) https://doi.org/10.1016/j.aop.2014.06.013

[4] Cirac, J.I., Pérez-García, D., Schuch, N., Verstraete, F.: Matrix product states and projected entangled pair states: Concepts, symmetries, theorems. Reviews of Modern Physics **93**(4), 045003 (2021) https://doi.org/10.1103/RevModPhys.93.045003

[5] Vidal, G.: Class of quantum many-body states that can be efficiently simulated. Physical Review Letters **101**(11), 110501 (2008) https://doi.org/10.1103/PhysRevLett.101.110501

[6] Evenbly, G., Vidal, G.: Tensor network renormalization. Physical Review Letters **115**(18), 180405 (2015) https://doi.org/10.1103/PhysRevLett.115.180405

[7] Kolda, T.G., Bader, B.W.: Tensor decompositions and applications. SIAM Review **51**(3), 455–500 (2009) https://doi.org/10.1137/07070111X

[8] Uschmajew, A., Vandereycken, B.: The geometry of algorithms using hierarchical tensors. Linear Algebra and its Applications **439**(1), 133–166 (2013) https://doi.org/10.1016/j.laa.2013.03.016

[9] Orús, R.: Tensor networks for complex quantum systems. Nature Reviews Physics **1**(9), 538–550 (2019) https://doi.org/10.1038/s42254-019-0086-7

[10] Bachmayr, M., Schneider, R., Uschmajew, A.: Tensor Networks and Hierarchical Tensors for the Solution of High-Dimensional Partial Differential Equations. Foundations of Computational Mathematics **16**(6), 1423–1472 (2016) https://doi.org/10.1007/s10208-016-9317-9

[11] Dektor, A., Rodgers, A., Venturi, D.: Rank-Adaptive Tensor Methods for High-Dimensional Nonlinear PDEs. Journal of Scientific Computing **88**(2), 36 (2021) https://doi.org/10.1007/s10915-021-01539-3

[12] Cichocki, A.: Tensor networks for big data analytics and large-scale optimization problems. arXiv preprint arXiv:1407.3124 (2014)

[13] Stoudenmire, E.M.: Learning relevant features of data with multi-scale tensor networks. Quantum Science and Technology **3**(3), 034003 (2018) https://doi.org/10.1088/2058-9565/aaba1a

[14] Zangrando, E., Schotthöfer, S., Ceruti, G., Kusch, J., Tudisco, F.: Geometry-aware training of factorized layers in tensor tucker format. Advances in Neural Information Processing Systems **37**, 129743–129773 (2024)

[15] Schollwöck, U.: The density-matrix renormalization group in the age of matrix product states. Annals of Physics **326**(1), 96–192 (2011) https://doi.org/10.1016/j.aop.2010.09.012

[16] Cirac, J.I., Pérez-García, D., Schuch, N., Verstraete, F.: Matrix product states and projected entangled pair states: Concepts, symmetries, theorems. Reviews of Modern Physics **93**(4), 045003 (2021) https://doi.org/10.1103/RevModPhys.93.045003

[17] Evenbly, G., Vidal, G.: Algorithms for entanglement renormalization. Physical Review B **79**(14), 144108 (2009) https://doi.org/10.1103/PhysRevB.79.144108

[18] Evenbly, G., Vidal, G.: Class of highly entangled many-body states that can be efficiently simulated. Physical Review Letters **112**(24), 240502 (2014) https://doi.org/10.1103/PhysRevLett.112.240502

[19] Karrasch, C., Bardarson, J.H., Moore, J.E.: Reducing the numerical effort of finite-temperature density matrix renormalization group calculations. New Journal of Physics **15**(8), 083031 (2013) https://doi.org/10.1088/1367-2630/15/8/083031

[20] Hauschild, J., Leviatan, E., Bardarson, J.H., Altman, E., Zaletel, M.P., Pollmann, F.: Finding purifications with minimal entanglement. Physical Review B **98**(23), 235163 (2018) https://doi.org/10.1103/PhysRevB.98.235163

[21] Zaletel, M.P., Pollmann, F.: Isometric Tensor Network States in Two Dimensions. Physical Review Letters **124**(3), 037201 (2020) https://doi.org/10.1103/PhysRevLett.124.037201

[22] Lin, S.-H., Zaletel, M.P., Pollmann, F.: Efficient simulation of dynamics in two-dimensional quantum spin systems with isometric tensor networks. Physical Review B **106**(24), 245102 (2022) https://doi.org/10.1103/PhysRevB.106.

245102

[23] Sappler, B., Kawano, M., Zaletel, M.P., Pollmann, F.: Diagonal isometric form for tensor product states in two dimensions (arXiv:2507.08080) (2025) https://doi.org/10.48550/arXiv.2507.08080

[24] Slagle, K.: Fast Tensor Disentangling Algorithm. SciPost Physics **11**(3), 056 (2021) https://doi.org/10.21468/SciPostPhys.11.3.056

[25] Aizpurua, B., Jahromi, S.S., Singh, S., Orus, R.: Quantum large language models via tensor network disentanglers. arXiv preprint arXiv:2410.17397 (2024)

[26] Krumnow, C., Veis, L., Legeza, Ö., Eisert, J.: Fermionic orbital optimization in tensor network states. Physical Review Letters **117**(21), 210402 (2016) https://doi.org/10.1103/PhysRevLett.117.210402

[27] Dektor, A., Venturi, D.: Tensor rank reduction via coordinate flows. Journal of Computational Physics **491**, 112378 (2023) https://doi.org/10.1016/j.jcp.2023.112378

[28] Dektor, A., Venturi, D.: Coordinate-Adaptive Integration of PDEs on Tensor Manifolds. Communications on Applied Mathematics and Computation **7**(4), 1562–1579 (2025) https://doi.org/10.1007/s42967-023-00357-8

[29] Hauru, M., Van Damme, M., Haegeman, J.: Riemannian optimization of isometric tensor networks. SciPost Physics **10**(2), 040 (2021) https://doi.org/10.21468/SciPostPhys.10.2.040

[30] Luchnikov, I., Ryzhov, A., Filippov, S., Ouerdane, H.: QGOpt: Riemannian optimization for quantum technologies. SciPost Physics **10**(3), 079 (2021) https://doi.org/10.21468/SciPostPhys.10.3.079

[31] Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information: 10th Anniversary Edition. Cambridge University Press (2010). https://doi.org/10.1017/CBO9780511976667

[32] Verstraete, F., Cirac, J.I.: Matrix product states represent ground states faithfully. Physical Review B **73**(9), 094423 (2006) https://doi.org/10.1103/PhysRevB.73.094423

[33] Recht, B., Fazel, M., Parrilo, P.A.: Guaranteed minimum-rank solutions of linear matrix equations via nuclear norm minimization. SIAM Review **52**(3), 471–501 (2010) https://doi.org/10.1137/070697835

[34] Lu, C., Tang, J., Yan, S., Lin, Z.: Nonconvex nonsmooth low rank minimization via iteratively reweighted nuclear norm. IEEE Transactions on Image Processing **25**(2), 829–839 (2015) https://doi.org/10.1109/TIP.2015.2511584

[35] Lewis, A.S., Sendov, H.S.: Nonsmooth Analysis of Singular Values. Part II: Applications. Set-Valued Analysis **13**(3), 243–264 (2005) https://doi.org/10.1007/s11228-004-7198-6 . Accessed 2025-08-21

[36] Löwner, K.: Über monotone Matrixfunktionen. Mathematische Zeitschrift **38**(1), 177–216 (1934) https://doi.org/10.1007/BF01170633

[37] Ding, C., Sun, D., Sun, J., Toh, K.-C.: Spectral operators of matrices. Mathematical Programming **168**(1), 509–531 (2018) https://doi.org/10.1007/s10107-017-1162-3

[38] Lewis, A.S., Sendov, H.S.: Nonsmooth Analysis of Singular Values. Part I: Theory. Set-Valued Analysis **13**(3), 213–241 (2005) https://doi.org/10.1007/s11228-004-7197-7

[39] Absil, P.-A., Mahony, R., Sepulchre, R.: Optimization Algorithms on Matrix Manifolds. Princeton University Press, Princeton, New Jersey (2007)

[40] Boumal, N.: An Introduction to Optimization on Smooth Manifolds. Cambridge University Press, Cambridge, UK (2023)

[41] Sappler, B.: Development of a diagonal isometric tensor network algorithm. Master's thesis, Technische Universität München (2024)

[42] Townsend, J.: Differentiating the Singular Value Decomposition. https://j-towns.github.io/papers/svd-derivative.pdf

[43] Sato, H.: Riemannian conjugate gradient methods: General framework and specific algorithms with convergence analyses. SIAM Journal on Optimization **32**(4), 2690–2717 (2022) https://doi.org/10.1137/21M1464178

[44] Fletcher, R., Reeves, C.M.: Function minimization by conjugate gradients. Comput. J. **7**, 149–154 (1964)

[45] Polak, E., Ribière, G.: Note sur la convergence de méthodes de directions conjuguées. ESAIM: Mathematical Modelling and Numerical Analysis - Modélisation Mathématique et Analyse Numérique **3**(R1), 35–43 (1969)

[46] Polyak, B.T.: The conjugate gradient method in extremal problems. USSR Computational Mathematics and Mathematical Physics **9**(4), 94–112 (1969) https://doi.org/10.1016/0041-5553(69)90035-4

[47] Hestenes, M.R., Stiefel, E.: Methods of conjugate gradients for solving linear systems. Journal of Research of the Bureau of National Standards **49**(6), 409–436 (1952)

[48] Dai, Y.H., Yuan, Y.: A nonlinear conjugate gradient method with a strong global convergence property. SIAM Journal on Optimization **10**(1), 177–182 (1999)

https://doi.org/10.1137/S1052623497318992

[49] Liu, Y., Storey, C.: Efficient generalized conjugate gradient algorithms, part 1: Theory. Journal of Optimization Theory and Applications **69**(1), 129–137 (1991) https://doi.org/10.1007/BF00940464

[50] Conn, A.R., Gould, N.I.M., Toint, P.L.: Trust Region Methods. Society for Industrial and Applied Mathematics, Philadelphia, PA (2000). https://doi.org/10.1137/1.9780898719857

[51] Steihaug, T.: The conjugate gradient method and trust regions in large scale optimization. SIAM Journal on Numerical Analysis **20**(3), 626–637 (1983) https://doi.org/10.1137/0720042

[52] Hurley, J.R., Cattell, R.B.: The procrustes program: Producing direct rotation to test a hypothesized factor structure. Behavioral Science **7**(2), 258–262 (1962) https://doi.org/10.1002/bs.3830070216

[53] Mickelin, O., Karaman, S.: On algorithms for and computing with the tensor ring decomposition. Numerical Linear Algebra with Applications **27**(3), 2289 (2020) https://doi.org/10.1002/nla.2289

[54] Townsend, J., Koep, N., Weichwald, S.: Pymanopt: A python toolbox for optimization on manifolds using automatic differentiation. Journal of Machine Learning Research **17**(137), 1–5 (2016)